

[illegible]

```

float vertices[65536];
unsigned int indices[17000];

int vIndex = 0;
int iIndex = 0;
int quadCount = 0;

float pixelSizeX = 0.033333f;
float pixelSizeY = 0.000001f;
float startX = -1.0f;
float startY = 1.0f;

for (int i = 0; i < 45; i++) {
    for (int j = 0; j < 60; j++) {
        if (per[4][j] != 0) {
            float r = 0.0f, g = 0.0f, b = 0.0f;

            if (per[4][j] == 1) { r = 0.0f; g = 0.0f; b = 0.0f; }
            else if (per[4][j] == 2) { r = 0.0f; g = 0.6f; b = 1.0f; }
            else if (per[4][j] == 3) { r = 0.9f; g = 0.1f; b = 0.1f; }
            else if (per[4][j] == 4) { r = 0.0f; g = 0.3f; b = 0.0f; }

            float x = startX + (j * pixelSizeX);
            float y = startY - (i * pixelSizeY);

            vertices[vIndex++] = x; vertices[vIndex++] = y; vertices[vIndex++] = 0.0f;
            vertices[vIndex++] = r; vertices[vIndex++] = g; vertices[vIndex++] = b;

            vertices[vIndex++] = x + pixelSizeX; vertices[vIndex++] = y; vertices[vIndex++] = 0.0f;
            vertices[vIndex++] = r; vertices[vIndex++] = g; vertices[vIndex++] = b;

            vertices[vIndex++] = x + pixelSizeX; vertices[vIndex++] = y - pixelSizeY; vertices[vIndex++] = 0.0f;
            vertices[vIndex++] = r; vertices[vIndex++] = g; vertices[vIndex++] = b;

            vertices[vIndex++] = x; vertices[vIndex++] = y - pixelSizeY; vertices[vIndex++] = 0.0f;
            vertices[vIndex++] = r; vertices[vIndex++] = g; vertices[vIndex++] = b;

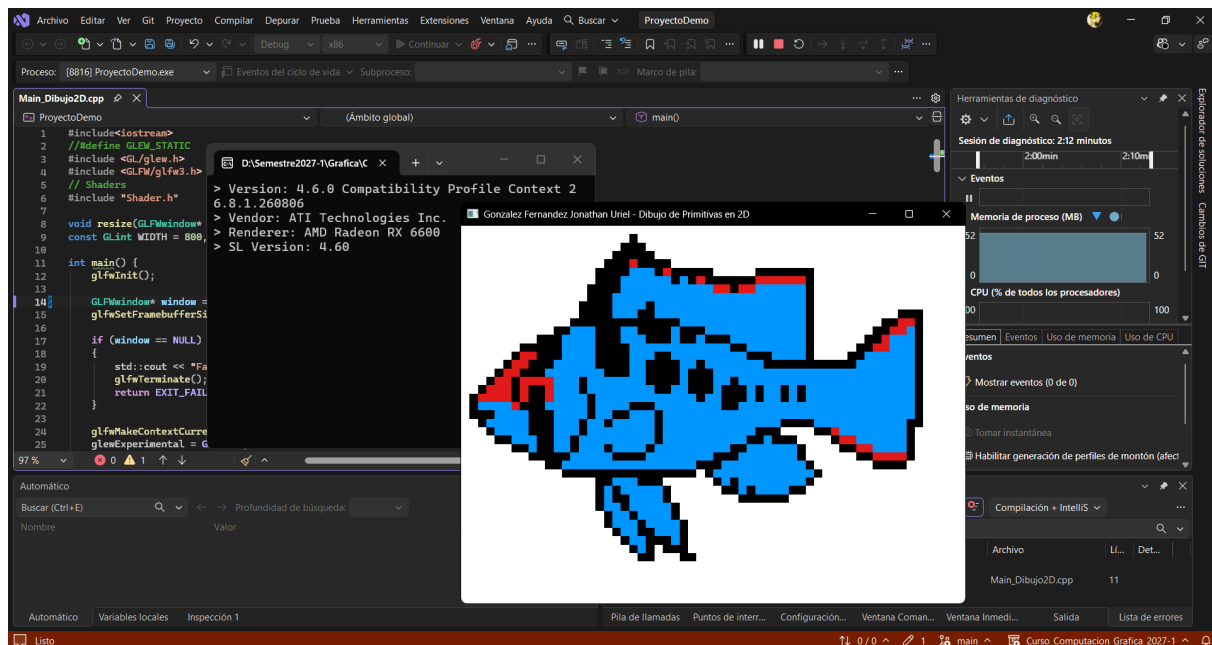
            int base = quadCount + 4;
            indices[iIndex++] = base;
            indices[iIndex++] = base + 1;
            indices[iIndex++] = base + 2;
            indices[iIndex++] = base + 2;
            indices[iIndex++] = base + 3;
            indices[iIndex++] = base;

            quadCount++;
        }
    }
}

```

El barrido de la matriz se ejecuta mediante dos bucles for anidados que inspeccionan las 45

filas y 60 columnas secuencialmente, utilizando una condición `if (pez[i][j] != 0)` para ignorar los espacios vacíos y optimizar el rendimiento de la memoria. Al detectar una celda con color, el algoritmo traduce sus índices (`i, j`) a coordenadas físicas (`x, y`) de OpenGL calculando la posición exacta en el eje X (`startX + (j * pixelSizeX)`) e invirtiendo matemáticamente el eje Y (`startY - (i * pixelSizeY)`) para coincidir con el plano cartesiano de la pantalla. A partir de esta coordenada de origen, se calculan las cuatro esquinas del píxel y se empujan secuencialmente sus datos espaciales, de color y los seis índices geométricos hacia los arreglos unidimensionales para que la tarjeta gráfica pueda ensamblar los dos polígonos correspondientes.



Salida final al ejecutar el programa.

Conclusión

Comprendí que realizar este dibujo únicamente con primitivas es mucho más tedioso, al ir colocando cada vértice me percataba que se iba alargando cada vez más el código y me estaba costando mucho tiempo. Por eso se decidió buscar una alternativa con ayuda de la IA. Además me di cuenta que en ocasiones me costaba integrar las demás primitivas de unión de vértices y triángulos. Considero que es una excelente forma de entender cómo es que funcionan los vértices y primitivas porque vas observando que distancia tiene que poner el vértice para que no se empalmen o se separen demasiado.

Referencias

Google. (2026). *Gemini* <https://share.gemini.google/AZ6seLclFrGK>
https://github.com/jonathanGF/Curso_Computacion_Grafica_2027-1